day11-computer-vision-1

March 13, 2026

[ ]:
```
# Install OpenCV for Python
!pip install opencv-python

# Verify installation
import cv2
print(cv2.__version__)   # e.g. 4.9.0
```

```
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-
packages (4.13.0.92)
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.12/dist-
packages (from opencv-python) (2.0.2)
4.13.0
```

[ ]:
```
#  Scenario: You received a JPG from a drone camera. Before processing,␣
 ↪understand its
#  dimensions, color space, and pixel type.

import cv2
import numpy as np
from google.colab.patches import cv2_imshow

#   Read the image (BGR by default)
img = cv2.imread('drone_photo.jpg')

# Check if image was loaded successfully
if img is None:
    print("Error: Could not load image. Please ensure 'drone_photo.jpg' exists␣
 ↪and the path is correct.")
else:
    #   Inspect properties
    print('Shape :', img.shape)     # (height, width, channels)
    print('Dtype :', img.dtype)     # uint8  →  0-255 per channel
    print('Size  :', img.size)      # total number of pixels

    #   Display
    cv2_imshow(img)
    # cv2.waitKey(0)                  # press any key to close
```

```
    # cv2.destroyAllWindows()

    #   Convert BGR → RGB (for matplotlib)
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
Shape : (757, 1162, 3)
Dtype : uint8
Size  : 2638902
```

[ ]:
```
import cv2

img = cv2.imread('images.jpg')

#   Resize
resized = cv2.resize(img, (224, 224))             # fixed WxH
half    = cv2.resize(img, None, fx=0.5, fy=0.5)  # 50% scale

#   Crop (NumPy slicing)
roi = img[100:400, 200:600]   # [y1:y2, x1:x2]

#   Rotate 45° around centre
h, w = img.shape[:2]
M   = cv2.getRotationMatrix2D((w//2, h//2), 45, 1.0)
rot = cv2.warpAffine(img, M, (w, h))
```

```
#   Flip
flip_h = cv2.flip(img,  1)   # horizontal
flip_v = cv2.flip(img,  0)   # vertical

#   Colour conversions
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
hsv  = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

[ ]:
```
from google.colab.patches import cv2_imshow

cv2_imshow(img)
cv2_imshow(resized)
cv2_imshow(half)
cv2_imshow(roi)
cv2_imshow(rot)
cv2_imshow(flip_h)
cv2_imshow(flip_v)
cv2_imshow(gray)
cv2_imshow(hsv)

# cv2.waitKey(0)          # waitKey and destroyAllWindows are not needed with␣
 ↪cv2_imshow
# cv2.destroyAllWindows() # as cv2_imshow displays images directly in the␣
 ↪notebook output
```

[ ]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(12,8))

plt.subplot(2,3,1); plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)); plt.
 ↪title("Original"); plt.axis("off")
plt.subplot(2,3,2); plt.imshow(cv2.cvtColor(resized, cv2.COLOR_BGR2RGB)); plt.
 ↪title("Resized"); plt.axis("off")
plt.subplot(2,3,3); plt.imshow(cv2.cvtColor(rot, cv2.COLOR_BGR2RGB)); plt.
 ↪title("Rotated"); plt.axis("off")
```

```
plt.subplot(2,3,4); plt.imshow(cv2.cvtColor(flip_h, cv2.COLOR_BGR2RGB)); plt.
 ↪title("Flip H"); plt.axis("off")
plt.subplot(2,3,5); plt.imshow(gray, cmap="gray"); plt.title("Gray"); plt.
 ↪axis("off")
plt.subplot(2,3,6); plt.imshow(cv2.cvtColor(hsv, cv2.COLOR_BGR2RGB)); plt.
 ↪title("HSV"); plt.axis("off")

plt.show()
```

[ ]:
```
#   Scenario:
# We have a scanned document (like a report or form).
# Scanned documents usually contain noise, shadows, and uneven lighting.
# Before performing OCR (Optical Character Recognition),
# we clean the image so that text becomes clearer.

# Import OpenCV library
# OpenCV is used for image processing and computer vision tasks
import cv2

# Import NumPy
# NumPy is used for numerical operations and creating matrices (like kernels)
import numpy as np
from google.colab.patches import cv2_imshow
```

```python
#
# Step 1: Read the image from disk
#

# cv2.imread() loads an image file from the specified path
# 'report.jpg' should exist in the working directory
img = cv2.imread('report.jpg')


#
# Step 2: Verify that the image was loaded correctly
#

# If the file path is wrong or the file does not exist,
# cv2.imread() returns None

if img is None:
    print("Error: Could not load image. Please ensure 'document.jpg' exists and␣
 ↪the path is correct.")

else:

    #
    # Step 3: Convert image to grayscale
    #

    # OCR usually works better on grayscale images
    # because color information is not required for text detection

    # cv2.cvtColor() converts color spaces
    # COLOR_BGR2GRAY converts a BGR image to grayscale

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)



    # ==========================================================
    #   Smoothing / Denoising
    # ==========================================================
    # Real scanned documents contain noise such as:
    # • scanner artifacts
    # • dust spots
    # • salt-and-pepper noise
    # • uneven lighting

    # Smoothing reduces noise before thresholding.
```

```python
# ---------------------------------------------------------
# Gaussian Blur
# ---------------------------------------------------------

# cv2.GaussianBlur(image, kernel_size, sigma)

# kernel_size = (5,5) → size of blur window
# sigma = 0 → OpenCV automatically calculates it

# Gaussian blur smooths the image by averaging nearby pixels
# using a Gaussian distribution.

blur_g = cv2.GaussianBlur(gray, (5, 5), 0)



# ---------------------------------------------------------
# Median Blur
# ---------------------------------------------------------

# cv2.medianBlur(image, kernel_size)

# Median blur replaces each pixel value
# with the MEDIAN of the neighboring pixels.

# This is extremely good at removing
# salt-and-pepper noise (random white/black pixels).

blur_m = cv2.medianBlur(gray, 5)



# ---------------------------------------------------------
# Bilateral Filter
# ---------------------------------------------------------

# cv2.bilateralFilter(image, diameter, sigmaColor, sigmaSpace)

# Unlike normal blurring:
# Bilateral filtering smooths the image
# BUT preserves edges.

# diameter = 9 → size of neighborhood
# sigmaColor = 75 → color similarity
# sigmaSpace = 75 → spatial closeness
```

```python
# This is useful when we want to reduce noise
# without destroying text edges.

blur_b = cv2.bilateralFilter(img, 9, 75, 75)



# ==========================================================
#   Sharpening using a Kernel
# ==========================================================

# A kernel (or filter matrix) is used to modify pixels.

# This sharpening kernel increases contrast around edges
# which makes text boundaries clearer.

# Kernel explanation:
#
#   0  -1   0
#  -1   5  -1
#   0  -1   0
#
# The center pixel gets multiplied by 5,
# while neighboring pixels are subtracted.
# This highlights edges.

kernel = np.array([
    [ 0, -1,  0],
    [-1,  5, -1],
    [ 0, -1,  0]
])


# cv2.filter2D applies the kernel to the image

# Parameters:
# src  → input image
# ddepth → output depth (-1 means same as input)
# kernel → convolution kernel

sharp = cv2.filter2D(gray, -1, kernel)



# ==========================================================
#   Thresholding
```

```
# ==========================================================
# Thresholding converts grayscale image → binary image

# This means:
# Text = white
# Background = black

# OCR engines prefer binary images because
# they clearly separate text from background.


# ----------------------------------------------------------
# Global Threshold
# ----------------------------------------------------------

# cv2.threshold(src, thresh, maxval, type)

# thresh = 127
# Pixels above 127 → set to 255 (white)
# Pixels below 127 → set to 0 (black)

# "_" is used to ignore the returned threshold value

_, thresh = cv2.threshold(
    blur_g,
    127,
    255,
    cv2.THRESH_BINARY
)



# ----------------------------------------------------------
# Adaptive Threshold
# ----------------------------------------------------------

# Useful when lighting is uneven across the document.

# Instead of using ONE threshold for the whole image,
# adaptive threshold calculates different thresholds
# for small regions.

adap_thresh = cv2.adaptiveThreshold(

    blur_g,                        # input image
    255,                           # max value (white)
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,# threshold method
```

```
    cv2.THRESH_BINARY,              # binary output
    11,                             # neighborhood block size
    2                               # constant subtracted
)



# ==========================================================
#   Morphological Operations
# ==========================================================
# Morphological operations modify shapes in binary images.

# They are useful for:
# • removing small noise
# • filling gaps in text
# • improving OCR readability


# ----------------------------------------------------------
# Create Morphological Kernel
# ----------------------------------------------------------

# np.ones creates a matrix of ones

# This kernel will slide across the image
# to perform morphological operations.

kernel2 = np.ones((3, 3), np.uint8)



# ----------------------------------------------------------
# Erosion
# ----------------------------------------------------------

# Erosion removes white pixels from object boundaries.

# Effect:
# • Shrinks white areas
# • Removes small noise

eroded = cv2.erode(
    thresh,
    kernel2,
    iterations=1
)
```

```
# ---------------------------------------------------------
# Dilation
# ---------------------------------------------------------

# Dilation expands white regions.

# Effect:
# • thickens text
# • fills small holes

dilated = cv2.dilate(
    thresh,
    kernel2,
    iterations=1
)



# ---------------------------------------------------------
# Opening
# ---------------------------------------------------------

# Opening = Erosion followed by Dilation

# Removes small noise while preserving shape

opened = cv2.morphologyEx(
    thresh,
    cv2.MORPH_OPEN,
    kernel2
)



# ---------------------------------------------------------
# Closing
# ---------------------------------------------------------

# Closing = Dilation followed by Erosion

# Useful for filling small gaps inside letters.

closed = cv2.morphologyEx(
    thresh,
    cv2.MORPH_CLOSE,
```

```
    kernel2
)



# ==========================================================
#   Image Information
# ==========================================================

# img.size returns total number of elements in the image
# For a color image: width × height × channels

print('Size  :', img.size)



# ==========================================================
#   Display Image
# ==========================================================

# cv2_imshow() is used in Google Colab
# because cv2.imshow() does not work there.

cv2_imshow(img)



# cv2.waitKey(0)
# waits for a key press before closing the window

# cv2.destroyAllWindows()
# closes all OpenCV windows



# ==========================================================
#   Convert BGR → RGB
# ==========================================================

# OpenCV loads images in BGR format
# but libraries like matplotlib expect RGB format.

img_rgb = cv2.cvtColor(
    img,
    cv2.COLOR_BGR2RGB
)
```

```
Size  : 555972
```

[ ]:
```
#
# Import cv2_imshow for displaying images in Google Colab
#

# In normal Python environments we use:
# cv2.imshow()

# However, cv2.imshow() does NOT work in Google Colab
# because Colab runs in a remote notebook environment.

# Google provides a special function called cv2_imshow()
```

```
# that correctly renders images inside notebook cells.

from google.colab.patches import cv2_imshow



#
# Display the Original Image
#

# This shows the raw scanned document before any processing.
# It helps us visually inspect:
# • noise
# • shadows
# • lighting issues
# • skewed or blurry text

cv2_imshow(img)



#
# Display the Grayscale Image
#

# Grayscale removes color information and keeps only intensity values.

# Why this matters:
# OCR algorithms usually work on grayscale or binary images
# because color does not contribute to recognizing text.

# Pixel values now range from:
# 0   → black
# 255 → white

cv2_imshow(gray)



#
# Display Gaussian Blurred Image
#

# Gaussian blur smooths the image by averaging nearby pixels.

# This reduces:
# • scanner noise
```

```
# • small pixel variations
# • minor artifacts

# The text edges become slightly softer,
# but noise in the background is reduced.

cv2_imshow(blur_g)



#
# Display Median Blurred Image
#

# Median blur replaces each pixel with the median value
# of surrounding pixels.

# This is especially good for removing:
# • salt-and-pepper noise
# (random black and white dots in scanned documents)

# Text edges remain relatively preserved.

cv2_imshow(blur_m)



#
# Display Sharpened Image
#

# Sharpening enhances edges by increasing contrast
# between neighboring pixels.

# The sharpening kernel emphasizes text boundaries,
# making letters more distinct from the background.

# This is helpful before thresholding
# because clearer edges produce cleaner binary text.

cv2_imshow(sharp)



#
# Display Global Threshold Image
#
```

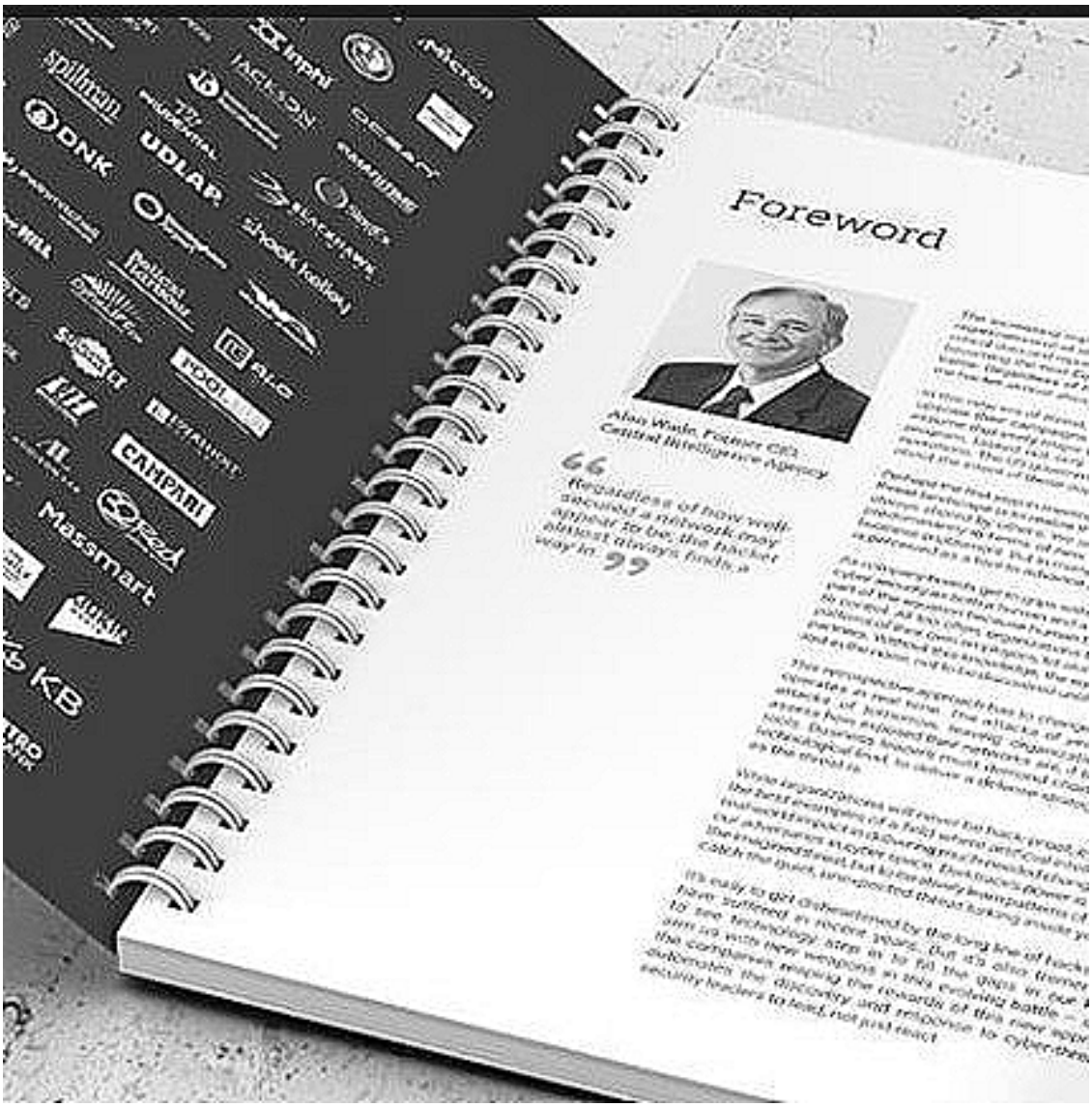
```
# Thresholding converts the grayscale image
# into a binary (black and white) image.

# In global thresholding:
# One fixed threshold value (127) is used
# for the entire image.

# Pixel rule:
# value > 127 → white
# value   127 → black

# This works well if lighting across the document
# is uniform.

cv2_imshow(thresh)




#
# Display Adaptive Threshold Image
#

# Adaptive thresholding calculates a different threshold
# for different regions of the image.

# This helps when the document has:
# • uneven lighting
# • shadows
# • faded areas

# Instead of one global value,
# thresholds are calculated locally.

cv2_imshow(adap_thresh)




#
# Display Eroded Image
#

# Erosion shrinks white areas in a binary image.

# Effects:
# • removes small noise
# • thins characters slightly
```

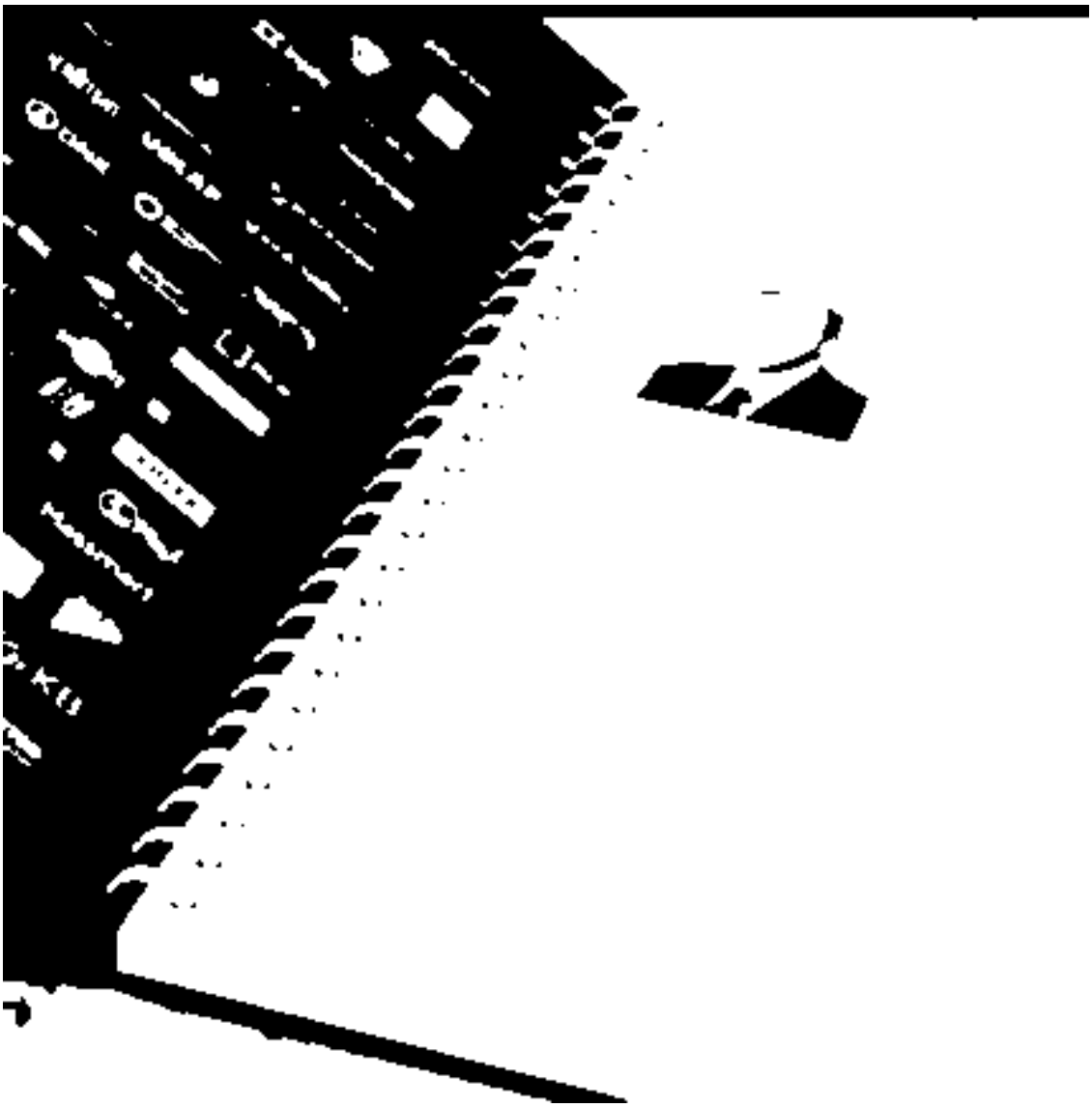
```
# • eliminates tiny white dots

# Useful when background noise appears as small white spots.

cv2_imshow(eroded)



#
# Display Dilated Image
#

# Dilation expands white regions.

# Effects:
# • thickens characters
# • fills small holes in letters
# • connects broken strokes

# Often used after erosion to restore main shapes.

cv2_imshow(dilated)



#
# Display Opening Operation
#

# Opening = Erosion followed by Dilation

# Purpose:
# Remove small noise without damaging
# the main structure of the text.

# Example improvement:
# tiny noise → removed
# letters → preserved

cv2_imshow(opened)



#
# Display Closing Operation
#
```

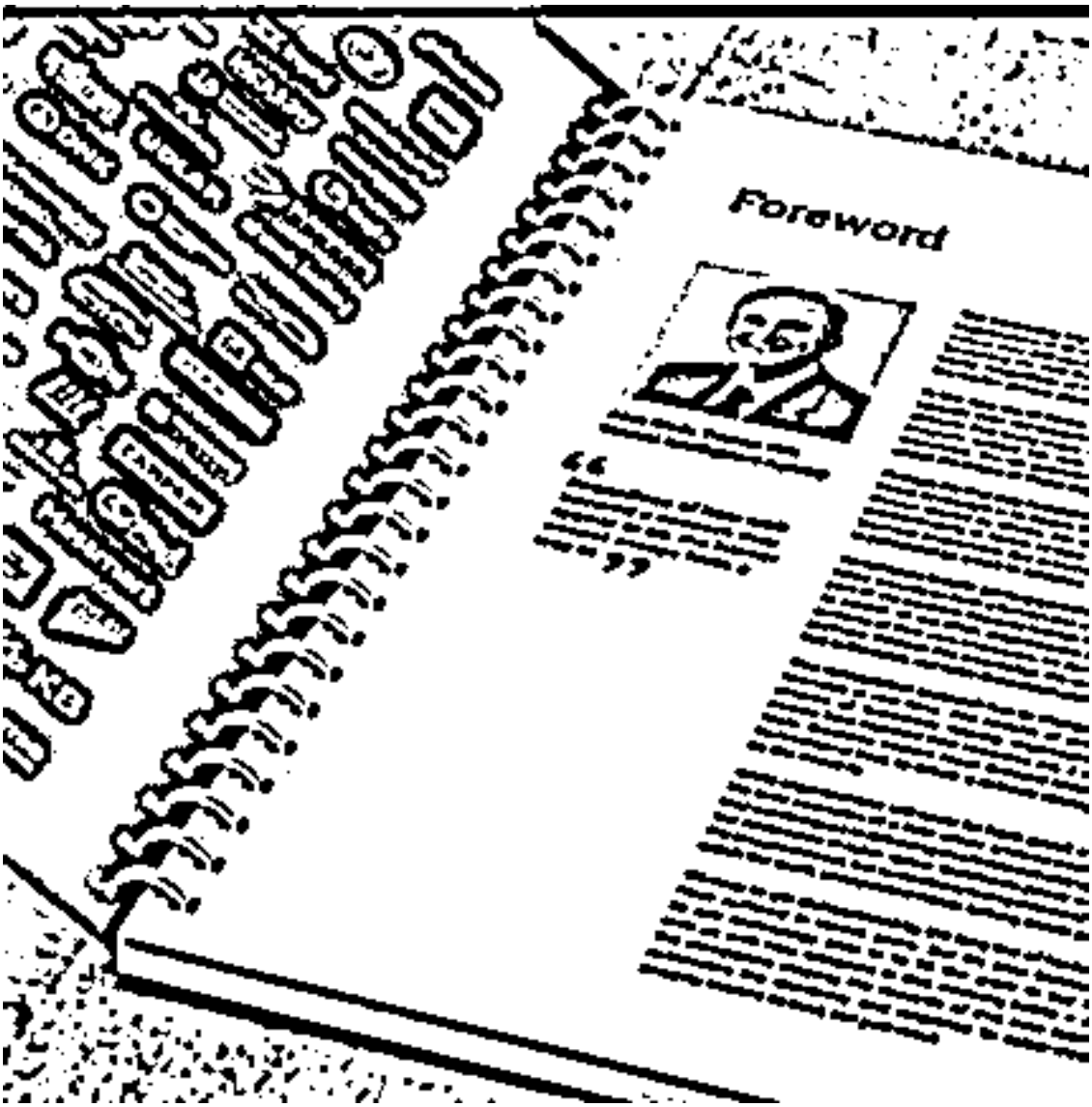
```
# Closing = Dilation followed by Erosion

# Purpose:
# Fill small gaps inside characters.

# Example improvements:
# broken letters → fixed
# small holes in characters → filled

cv2_imshow(closed)
```

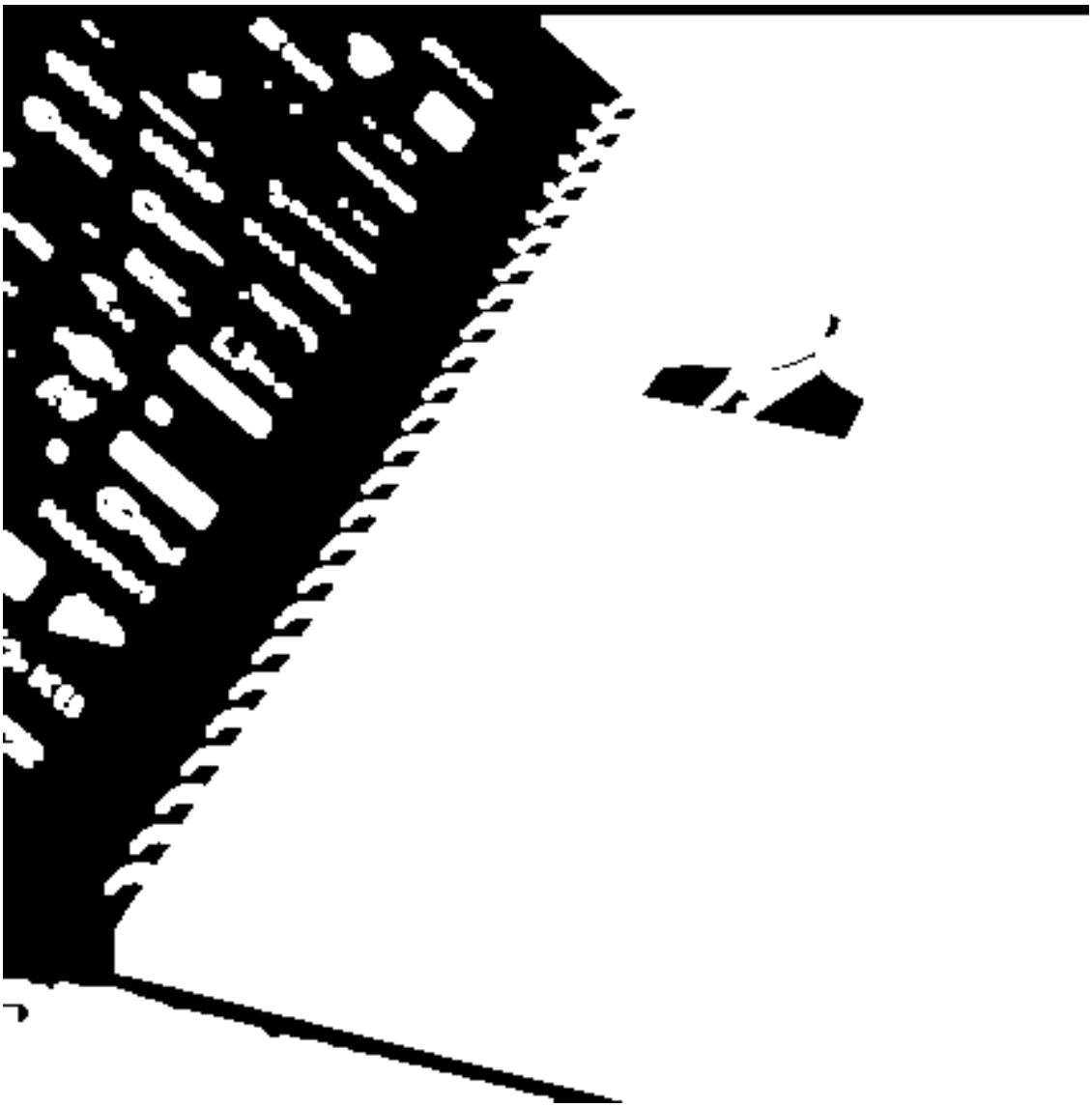
Foreword
Alan Wade, Former CIO, Central Intelligence Agency
"Regardless of how well-secured a network may appear to be, the hacker almost always finds a way in."
Spillman
DNK
UDLAR
Massmart
KB
CAMPARI

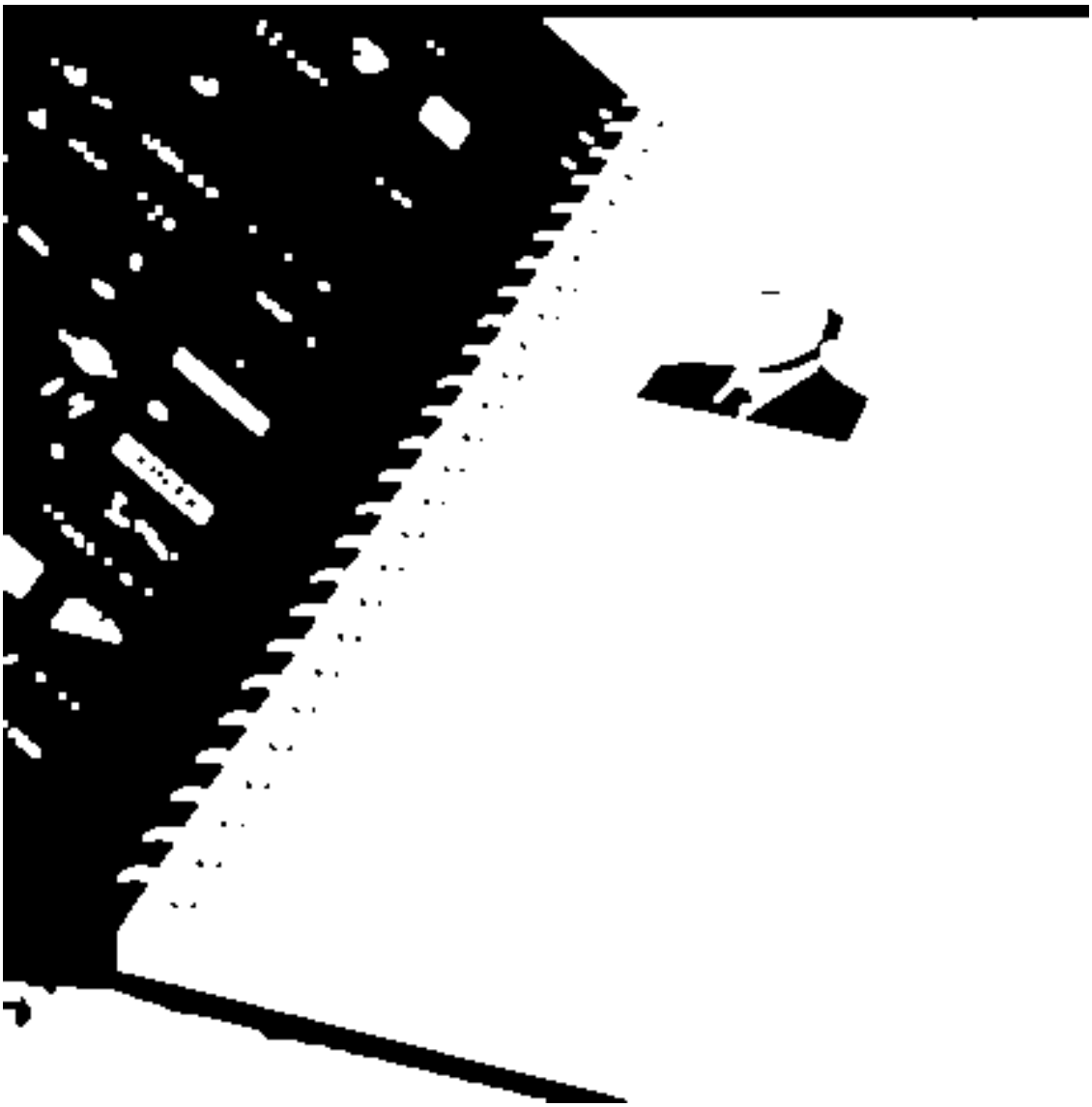
Foreword

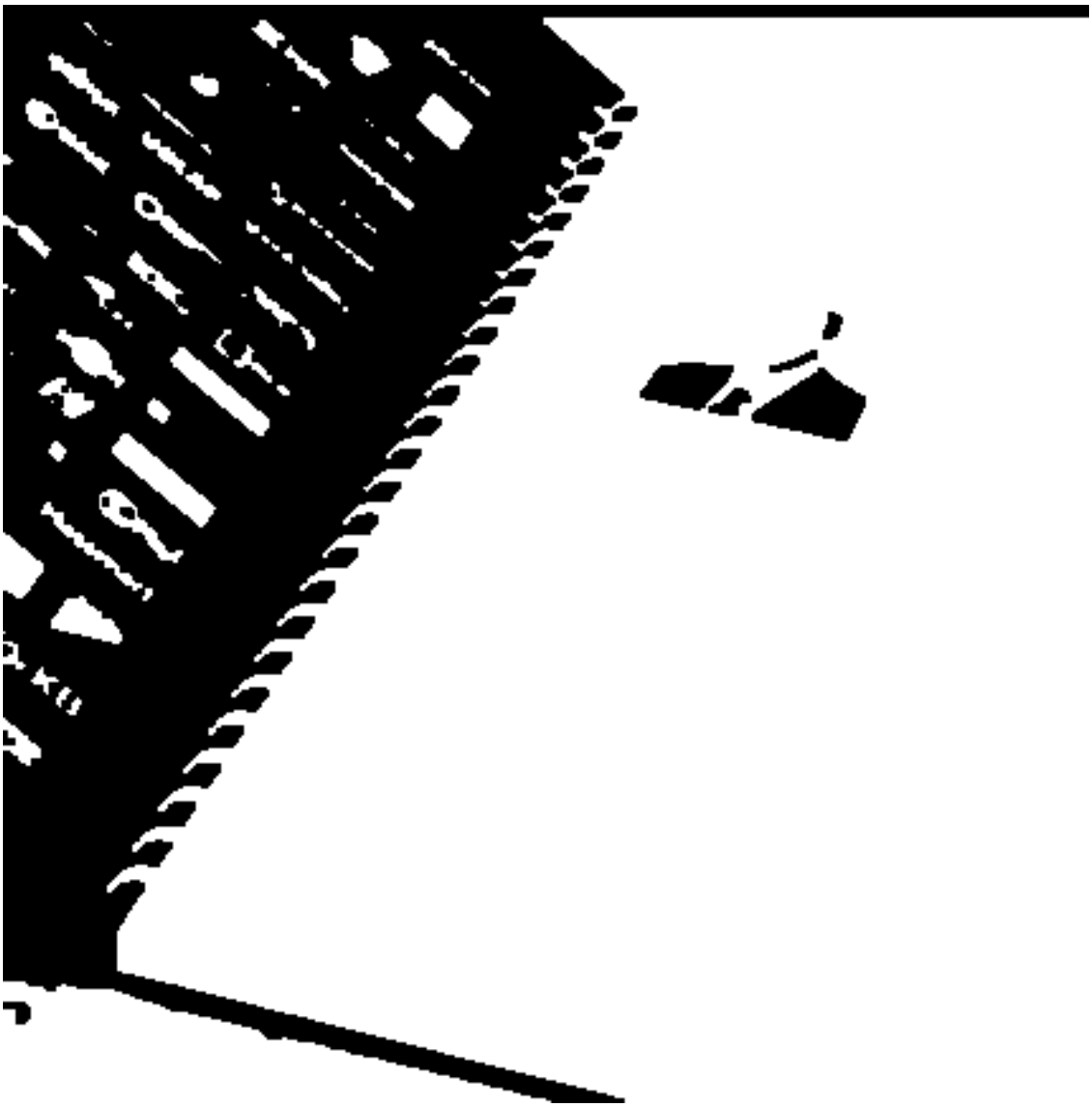

Foreword
Alan Wade, former CIO,
Central Intelligence Agency
"Regardless of how well-secured a network may appear to be, the hacker almost always finds a way in."




Foreword







[2]:
```
# Upload image first
from google.colab import files
uploaded = files.upload()

import cv2
import numpy as np
from google.colab.patches import cv2_imshow

# Step 1: Load image
frame = cv2.imread('road.jpeg')

if frame is None:
```

```
    print("Image not found. Make sure road.jpg is uploaded.")
else:

    # Step 2: Convert to grayscale
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Step 3: Gaussian blur
    blurred = cv2.GaussianBlur(gray, (5,5), 0)

    # Step 4: Canny edge detection
    edges = cv2.Canny(blurred, 50, 150)

    # Step 5: Region of Interest
    h, w = edges.shape
    mask = np.zeros_like(edges)

    pts = np.array([[0,h],[w,h],[int(w*0.6),int(h*0.6)],[int(w*0.4),int(h*0.
 ↪6)]], np.int32)
    cv2.fillPoly(mask, [pts], 255)

    roi = cv2.bitwise_and(edges, mask)

    # Step 6: Overlay result
    edges_col = cv2.cvtColor(roi, cv2.COLOR_GRAY2BGR)
    result = cv2.addWeighted(frame, 0.8, edges_col, 1, 0)

    cv2_imshow(result)
```

```
<IPython.core.display.HTML object>
```

```
Saving road.jpeg to road (2).jpeg
```